

Instructor: Date:

Learner: Regional Branch:

Program: Center:

CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

What are POP and OOP about?



Learning Outcome:

Talk about principles, concepts, characteristics of general-purpose computer programming language in spoken and written situations.

Ask your partner:

- What do you like about program writing? Why?
- Do you think all people can write programs? Why? Why not?

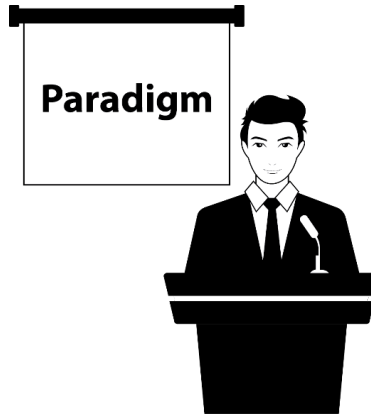
Can Do Statements:

- I can say what a programming paradigm is.
- I can talk about characteristics on different types of programming characteristics.
- I can discuss the differences between procedural paradigms and object oriented paradigms.



Let's Explore!

An expert talks about paradigms in programming.

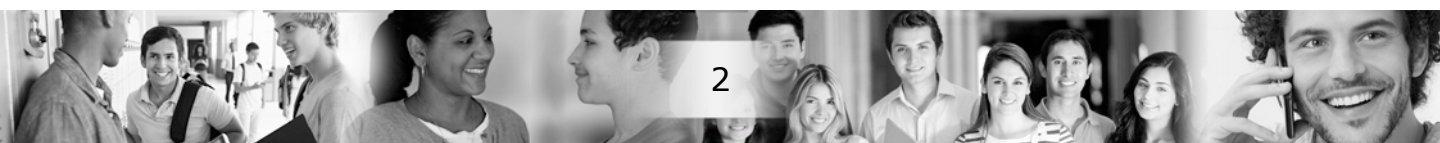


Instructions: Read the text below.

Paradigms in programming are going to remain essential for a very long time. There are two existing approaches to writing a program: procedure-oriented programming (POP) and object-oriented programming (OOP). You ought to write a program using the paradigm that best fits your experience and knowledge, yet there are differences between both approaches. As a professional, you will always want to know what those differences are. It doesn't matter if you might want to focus on object-oriented programming as it is the most popular paradigm nowadays.

From the moment computers were invented, many different ways of programming have been tried out: top-down programming, bottom-up programming, modular programming, structured programming, etc. They all have specific and basic purposes to make programming efficient. What will you be expecting from such approaches? Well, they should make the language for writing a program easier, bug-free, less hard to understand, easily modifiable, etc.

As a programmer, I can say you had better learn some aspects that help you to differentiate POP from OOP and its basic units. There's a set of instructions to be followed step by step. You are going to be facing various levels of difficulty regarding program writing for our class. Here are some facts: For developing a fairly complex program, you will choose a procedural approach (POP). If you are good at programming with POP, you might want to do the same with a more complex program, but actually, you are going to be working with OOP. OOP allows collaborative working, which is what we are looking for in our projects for next year. The main reason for this is code reusability; you work on a particular piece of code for next week, but one of your programmers can reuse it as many times as he wants for the oncoming projects. This makes software development much faster and efficient.



**Let's Practice!**

Those are some facts that expert mentioned about paradigms.

Instructions: Complete the sentences by writing its corresponding word. Use capital letter where necessary. Use: bug-free, modifiable, procedure-oriented, languages, object- oriented.

1. There are plenty of programming _____, but paradigms provide a way to classify them.
2. Nowadays, programing paradigms can make the process of designing a program be easily _____.
3. Paradigms involve classifying languages, but all of them must be _____.
4. _____ programming could be the most popular paradigm now.
5. _____ programming is used to develop complex programs.

**Let's Practice!**

Summarizing Some of the Recommendations an Expert Provide

Instructions: Read the paragraph. Underline all the mistakes in the sentences.

1. You might want to writing a program using the paradigm that best fits your experience and knowledge.
2. It doesn't matter if you are want to focus on object-oriented or procedure-oriented programming.
3. You may want to know that there are instructions to be followed step by step , using basic units of a language.
4. As a programmer, you would like to learn some aspects that differentiate POP and OOP.
5. You might want too reuse code while you are working on a particular section of the whole code.



Extension Activity

Describing Paradigms for Each Situation

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Look for further information about POP and OOP write down notes.
- Create a chart in which you show the main features of each one in specific situations. Provide examples.
- Share with your partners.

Self-Assessment

Yes

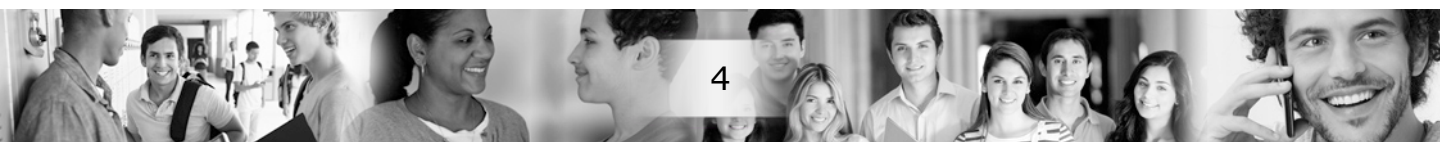
No

- I look for information about POP and OOP.
- I create a chart to show the main features of each paradigm.
- I talk about programming paradigms.

☐ ☐☐ ☐☐ ☐

Target Words

- **modifiable** (adjective): able to be changed or altered
e.g. Well, they should make the language for writing a program easier, bug-free, less hard to understand, easily modifiable, etc.
- **bug-free** (adjective): with no code errors
e.g. A programming approach should make the process of writing a program easier, easily modifiable, and bug-free.
- **paradigm** (noun): a model or a set of perspectives to understand and define a topic or reality
e.g. Paradigms in programming are going to remain essential for a very long time.
- **procedural** (adjective): following a procedure
e.g. For developing a fairly complex program you will choose a procedural approach (POP).
- **step-by-step** (expression): in a way that follows a series of steps to complete a process
e.g. There's a set of instructions to be followed step by step.
- **face** (verb): to deal with
e.g. You are going to be facing various levels of difficulty regarding program writing for our class.





Learning Points

The Future With Will, Going to, the Present Continuous and the Present Simple

Let's review the following structures. Remember they are used in many different ways: You can use **simple present** and **present continuous** to talk about activities that take place in the present in general or that are in progress in the present. Nevertheless, **Simple present**, along with **Going to**, **present continuous** and **Will**, can also be used to talk about future actions. Let's see how they work!

Simple Present:

We use the **simple present** to talk about permanent habitual facts in the present, but it can also be used as part of a prospective time from the present; this means to refer to future actions. Let's remember the structure of the **simple present**:

subject + verb in simple present + complement

Examples:

*Programming processes **finish** next week.*

*My co-workers **have** to design several software apps for next month.*

Don't forget to add the **-s** or **-es** to the verb in the third person affirmative form.

Examples:

*The system **starts** working tomorrow at 08:00 a.m.*

For negative and interrogative forms in **simple present**, use the modal verb **do** which in the third person becomes **does**. The main verbs of the sentence keep their base form; we do not add the **-s** when we formulate negative or interrogative statements.

Examples:

***Do** we **have** the programming meeting tomorrow?*

*She **doesn't** finish writing new codes on time.*

Present Continuous:

We use the **present continuous** to talk about actions that are happening at the moment of speaking, or/and temporary situations; it is also used to talk about actions in the future.





Learning Points

Examples:

***I'm working** on my apps **over the weekend**.*

***Next year**, we're **developing** an amazing software system.*

The structure of the **present continuous** has the verb **be** and an action verb with **-ing** at the end. To make the negative statement, we just have to add the particle **not** next to the verb "**be**". Don't forget to conjugate the verb **be**.

Examples:

*We're **working** on the new software next month.*

*He's **not working** on that project until next year.*

To make questions in the **present continuous**, we place the verb "**be**" (conjugated) before the subject. Just as in the simple present, only a **w-h** word can come before the verb **be**.

Examples:

***Are** we working on the project next week?*

***What software** are we working on tomorrow morning?*

Future with Going To:

We use **be going to** to talk about plans in the future.

Examples:

***I'm going to** design a software next week.*

*We **are going to** visit the client to collect requirements next month.*

To make the negative and interrogative forms, we add **not** to the verb **be** conjugated. For questions, we put the verb **be** before the subject, just as we do in the present continuous.

Examples:

*We **are not going to** continue with that project.*

*Why **are you going to** quit the programming team?*

Future with Will:

We use the modal verb **will** to talk about plans in the future.

Examples:

*I **will design** an accounting software for IBC.*

*We **will visit** the client to collect requirements in the next months.*



Learning Points

The structure of sentences with **will** is quite similar to the simple present, with the difference that instead of using the auxiliary verb **do** or **does** we use **will**. To make the negative and interrogative forms, we add **not** after **will**. For questions, we place **will** before the subject, just as we do in the simple present.

Examples:

*We **will not** (**won't**) continue with that project next year.*

***Will** you work on that project for the next coming months?*

Examples

- *As a professional, you will always want to know what those differences are.*
- *What will you be expecting from such approaches?*
- *You are going to be facing various levels of difficulty regarding program writing for our class..*

Might Want to for What Is Advisable

Modal verbs are used to express possibility, obligation, advisability, request, necessity, and ability. Today, we will focus on **might want to** for advisability. As you can see, the modal verb **might** is part of a longer structure that includes the verb want with its usual preposition **to**. Let's see how this structure works!

Grammar Structure

To use **might want to** in a sentence, follow this structure:

Subject + might want to + verb + complement

Example:

- *I think they **might want to continue** the contract only if we complete the project on time.*

The case of **might want to** is very interesting because it permits to give a stronger emphasis to what you want to advise on, but with a high degree of politeness.

Examples:

- *Your team **might want to focus** on program design, since they need to finish that first.*
- *You **might want to study** all the possible scenarios, as it is important to foresee any flaws in the system.*





Learning Points

Examples

- *It doesn't matter if you might want to focus on object-oriented programming, as it is the most popular paradigm nowadays.*
- *If you are good at programming with POP, you might want to do the same with a more complex program.*